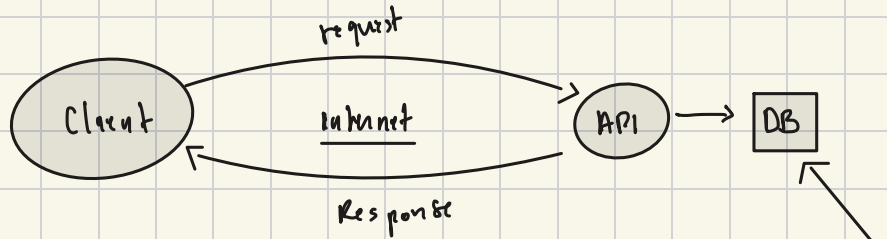




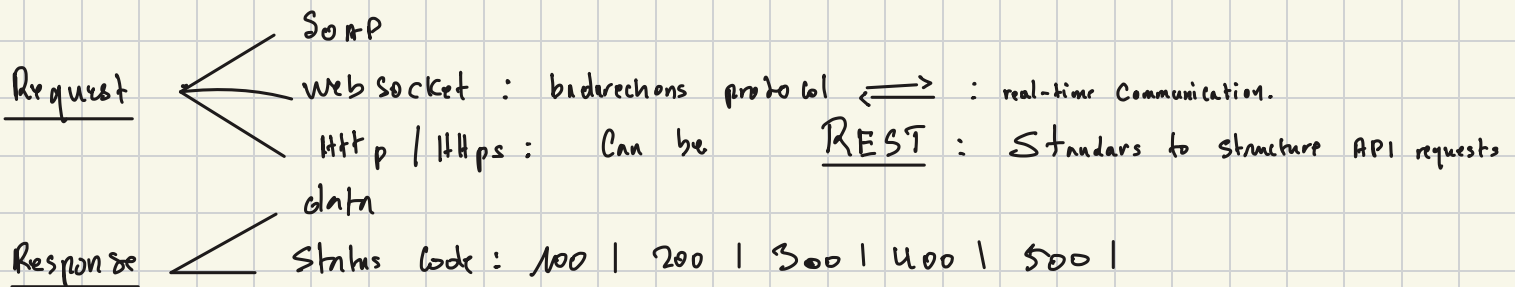
REST APIs

* Api = Software mechanism that simplifies dev by doing the following.

- abstracting implementation details.
- Exposing only objects or actions that a developer needs
- Establishing how an information provider and a information user communicate.



⇒ Api provides the interface to application resources such as database



API Gateway

→ act as Proxy between Client and Servers.

→ limit the request rate

→ allow or disallow requests by source.

→ Validate the dev Key.

→ Limit access by geographic area

→ abstract common functionality from the server

→ Reduce the number of the req that reach the server.

⇒ Similar to a door man. ⇒ act like a front door to access for app to access Data.

↳ it's a Serverless managed Service to build Api.

→ **Stateless**: chaque req est indépendante. le serveur ne se souvient de rien entre deux appels. ⇒ tout le contexte nécessaire doit être chaque req (token, permission, etc...) ⇒ Scalabilité: n'importe quel serveur peut répondre.

→ **Stateful**: le serveur sait qui communique et maintient l'état de la connexion tout le long.

→ State: toutes infos qui changent dans le temps pour un user: logged? Combien de temps?

- Api Gateway isole des fonctionnalités partagées que chaque serveur devrait gérer manuellement :
- ↳ AUTH (JWT ; IAM, lambda auth)
 - ↳ Throttling (rate limits) ; CORS ; Caching ; Stages (dev / Prod / Canary)

2 types of Api

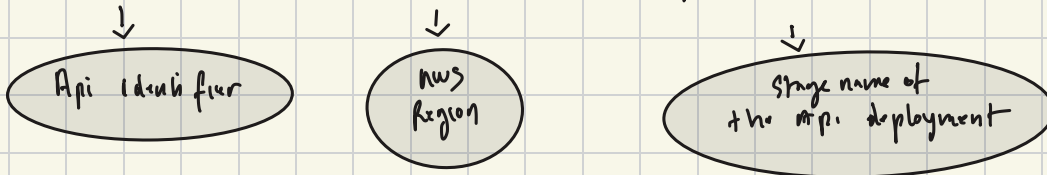
- **REST API** :
 - Give a developer full Control over the API req & Res
 - Support features that are not yet available in HTTP API's
- **HTTP API** :
 - Simplifies the dev of API's that require only API Proxy functionality.
 - Designed for the lowest cost and lowest latency.
- **Websocket** :
 - bidirectional communication.
 - for Secure encrypted version of web sockets => Secure Web Socket / WSS

Creating an API REST

- Use AWS CLI
- Use the API Gateway console
- ⇒ Base structure for REST and HTTP API's

Watch PDF

HTTPS: // {restapi-id}.execute-api.{region}.amazonaws.com/{stage-name}/



- ⇒ Practise :
- Get route
 - Put route
 - use the greedy path template {Proxy+}
 - Open API / Swagger => Have the doc of your API

Integrating API Gateway

(+) Endpoints types :

- Edge optimized : use its own Fully managed CloudFront distribution
 - ↳ give DDoS protection } Only available with RDS
- Regional endpoint : - For General - Recommended for building in a same region.
 - ↳ Best practice : Set up a cloudfront distribution in front of these endpoints. ⇒ Protect also against DDoS attack.

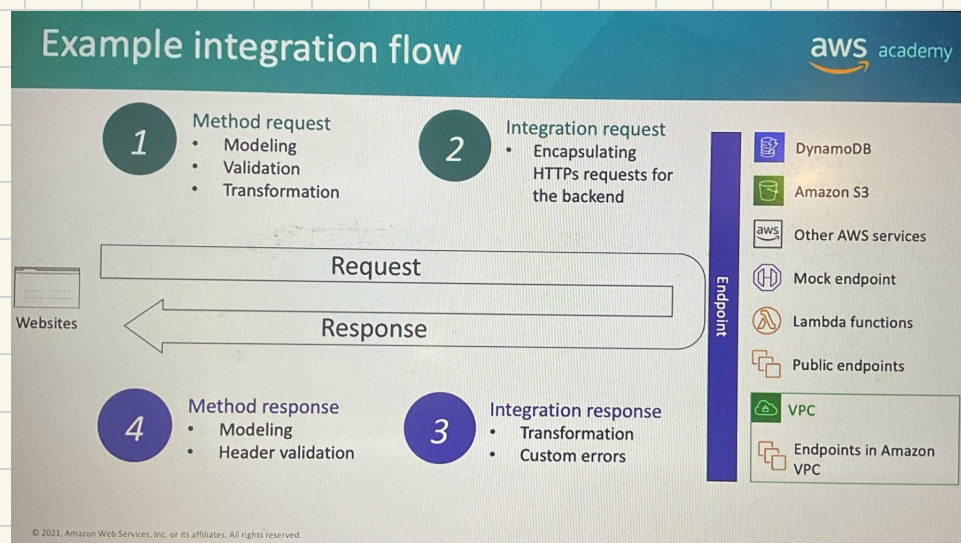
Private endpoint : Only accessible from VPC ⇒ For building Api that are used internally by Private micro services.

⇒ Can use API Gateway SDK.

(+) Integrations with others services

- Lambda
- DynamoDB.
- any other AWS service.
- Mock endpoints (can be swapped with a real backend)

⇒ API Gateway ne fait pas que router - il respecte, transforme et contrôle la req dans les deux sens entre le client et n'importe quel backend.



(+) Lambda non-proxy vs proxy integration.

⇒ Api gateway pass the wrapped request (all meta data lambda needs) to the target function. ⇒ the Response is handled differently → Proxy or non

Non - Proxy integration ⇒ Api Gateway transform the req → Send it to lambda.

↳ Configurations manuelles : validation, headers ---

↳ Contrôle total mais plus des Config ⇒ VTL

Proxy integration : le proxy passe tout directement sans transformer ou toucher à qlq chose

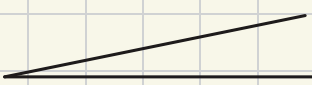
(+) First - class integrations ⇒ Connects an HTTP API to an AWS Service API
Use Case: Send a message to amazon
SQS.

⇒ You can use it to connect a publicly routable HTTP endpoint to an
API rule.

Use Case: migrate site by site a monolithique

⇒ Support transformations.

Section 6 : Deployments

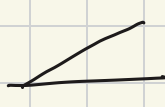
⇒ Multi stages available :  > the API must be associated to a stage

⇒ We can use Canary or blue green deployment.

⇒ We can improve the code

7. Controlling access to a REST API

1. IAM resources policies → attach the json to the API. ⇒ use to allow and disallow certain people.
2. Set up Certification :
 - Generate a client ssl by using API G.
 - the backend verifies the request that comes from API G. by using a public key.
 - Expires after 365 days.
3. AWS WAF : Protect the API from common web exploits like SQL injection and XSS. ; block req based on origine (IP ; country ; region agent ---)
 - Can match some regular Expression or string
 - Can be deployed on cloudfront as part of the content delivery network

4. CORS When Javascript is making a req (calling another website) from another website in the background the called website must block the request.
 - ⇒ Use appel JS : Cross-origina
 - ⇒ The pirate Can use browser cookies and sessions to make the request by Javascript to steal data.
 - ⇒ you can allow CORS  Specifies origins resources

5. le throttling : limiter le nombre de req pour protéger l'api contre la surcharge ou les attaques. ⇒ limiter à plusieurs niveaux
 - client + méthode
 - client
 - Méthode + stage
 - stage
 - account-wide : 10.000 req/sec per region / & appliquer en dernier.

le niveau le plus granulaire gagne.

+ Authorization and Authentication

1. None (Open) : Pas de sécurité } tout le monde
usage: Api public; webhooks; prototype.
2. IAM : le client doit avoir des credentials AWS (Access Key + Secret)
usage: Signer les requêtes avec credentials.
↳ Communication machine à machine dans AWS (lambda qui appelle Api G)
3. JWT (Cognito ou tiers) : le client envoie un token dans le header.
⇒ Api Gateway vérifie le token.
4. lambda authorizers : écrire une logique custom d'autorisation.

8. Monitoring

→ logs messages
→ details metrics.

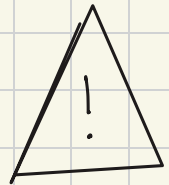
Send to cloudwatch

Other monitoring tools:

AWS X-ray

AWS Grafana

AWS CloudTrail



9. Optimization

→ Caching.

Cache hit count

Cache miss count

→ payload compression.

